Software Design Document (SDD)
Lab assignment 01: GPIO

GPIO Driver — STM32F411RE/F446RE

Module	**GPIO Driver**
Version	**1.0.0**
Team	**Adalberto Aguirre Mauricio Vela Steven McClellan**
Date	**27/04/2026**
Institution	**CETYS Universidad**

Introduction

Purpose

This document describes the design and architecture of the GPIO (General Purpose Input/Output) driver module for the STM32F411RE microcontroller, as developed in Lab Assignment 01. It provides details on data structures, hardware register mappings, control flow, and the public API to guide implementation and testing of the higher-level LED and Push-Button modules that rely on GPIO functionality.

Scope

The GPIO driver is a low-level software component for the STM32F411RE embedded system running on the NUCLEO-64 development board. It abstracts the GPIO hardware registers of the STM32F411RE microcontroller and provides a clean, validated API for higher-level modules to initialize GPIO ports, configure pin modes, and perform atomic read, write, and toggle operations on individual GPIO pins. The driver supports ports A through H and covers pins 0 through 15 on each port.

Definitions and Acronyms

Term	Definition
GPIO	General Purpose Input/Output — configurable digital pins on a microcontroller.
STM32F411RE	ARM Cortex-M4 based microcontroller by STMicroelectronics used in the NUCLEO-64 board.
RCC	Reset and Clock Control — peripheral responsible for enabling clocks to GPIO ports.
AHB1	Advanced High-performance Bus 1 — bus to which GPIO peripherals are connected on STM32F4.

MODER	GPIO Mode Register — configures each pin as input, output, alternate function, or analog.
ODR	Output Data Register — holds the output state of each GPIO pin.
IDR	Input Data Register — reflects the current logic level of each GPIO pin.
BSRR	Bit Set/Reset Register — allows atomic set and reset of individual GPIO output pins.
CMSIS	Cortex Microcontroller Software Interface Standard — vendor-independent hardware abstraction layer providing register definitions only. No HAL middleware is used.
SRS	Software Requirements Specification — document defining the functional requirements for this module.
SDD	Software Design Document — this document.
API	Application Programming Interface — set of public functions exposed by the driver.
FR	Functional Requirement — a requirement that specifies a system behavior.
NFR	Non-Functional Requirement — a requirement that specifies a quality attribute of the system.
HAL	Hardware Abstraction Layer — NOT used in this project. All hardware access is performed via direct bare-metal register manipulation.

Requirements

The table below summarizes the traceability between each functional requirement and the corresponding public API function implemented in this driver.

Req. ID	Function	Requirement Statement
FR-1	gpio_init()	The system shall configure all GPIO ports to a default state.
FR-2	gpio_initPort()	The system shall enable clocking for the specified port.
FR-3	gpio_setPinMode()	The system shall allow configuration of pin direction and mode.
FR-4	gpio_setPin()	The system shall drive the specified pin to a high state.
FR-5	gpio_clearPin()	The system shall drive the specified pin to a low state.
FR-6	gpio_togglePin()	The system shall invert the current state of the specified pin.
FR-7	gpio_readPin()	The system shall return the digital state of the specified pin.
FR-8	led_init()	The system shall initialize PA5 as a digital output for LED control.
FR-9	led_on() / led_off()	The LED module shall control the LED state using the GPIO API exclusively.
FR-10	led_toggle()	The system shall toggle the LED state without tracking it manually.
FR-11	button_init()	The system shall initialize PC13 as a digital input for button reading.
FR-12	button_get_state()	The system shall return the logical button state (PRESSED / RELEASED).

The following non-functional requirements are also applicable to this driver:

- NFR-1: The implementation shall be lightweight and efficient — no dynamic memory allocation and no HAL middleware.
- NFR-2: All functions shall validate input parameters (port range, pin range 0–15, mode range) and return a non-zero error code for invalid inputs.
- NFR-3: Atomic set and clear operations shall be performed using the BSRR register to prevent intermediate glitch states on output pins.

System Architecture

High-Level Design

The GPIO driver follows a layered software architecture that separates application logic from low-level hardware access. The three layers are described below.

Layer	Description
Application Layer	Higher-level software (main.c) that calls the GPIO driver public API functions to control hardware behavior without direct register access. Contains the main loop, edge-detection logic, and debounce strategy.
Driver Layer	Implements the public API declared in SRS_GPIO_DRIVER.h. Validates all input parameters, enables peripheral clocks via RCC, and performs direct read/write operations on GPIO hardware registers through the GPIO_arr[] pointer array.
Hardware Layer	Actual STM32F411RE GPIO peripheral registers (MODER, ODR, IDR, BSRR, etc.) mapped at fixed memory addresses via the AHB1 peripheral bus.

Module Interfaces

The GPIO driver exposes its functionality exclusively through the public header file SRS_GPIO_DRIVER.h. The module has the following interface characteristics:

- Public API: All user-callable functions are declared in SRS_GPIO_DRIVER.h. These include gpio_init, gpio_initPort, gpio_setPinMode, gpio_setPin, gpio_clearPin, gpio_togglePin, and gpio_readPin.
- Internal implementation: All function bodies and the internal static pointer array GPIO_arr[] are defined in SRS_GPIO_DRIVER.c and are not visible to external modules.
- Dependencies: The driver depends only on CMSIS register definitions for fixed-width integer types and hardware base addresses. No HAL or external libraries are used; all hardware access is performed through direct register manipulation.
- Hardware dependencies: STM32F411RE GPIO peripheral registers mapped at AHB1PERIPH_BASE + port-specific offsets, and the RCC AHB1ENR register for clock gating.

Class Diagram

The figure below represents the static structure of the GPIO driver, showing the register-map structures, the enumerations for port and mode, and their relationships to the three-layer architecture. Public members are marked with a filled circle (●) and private/internal members with an open circle (○).

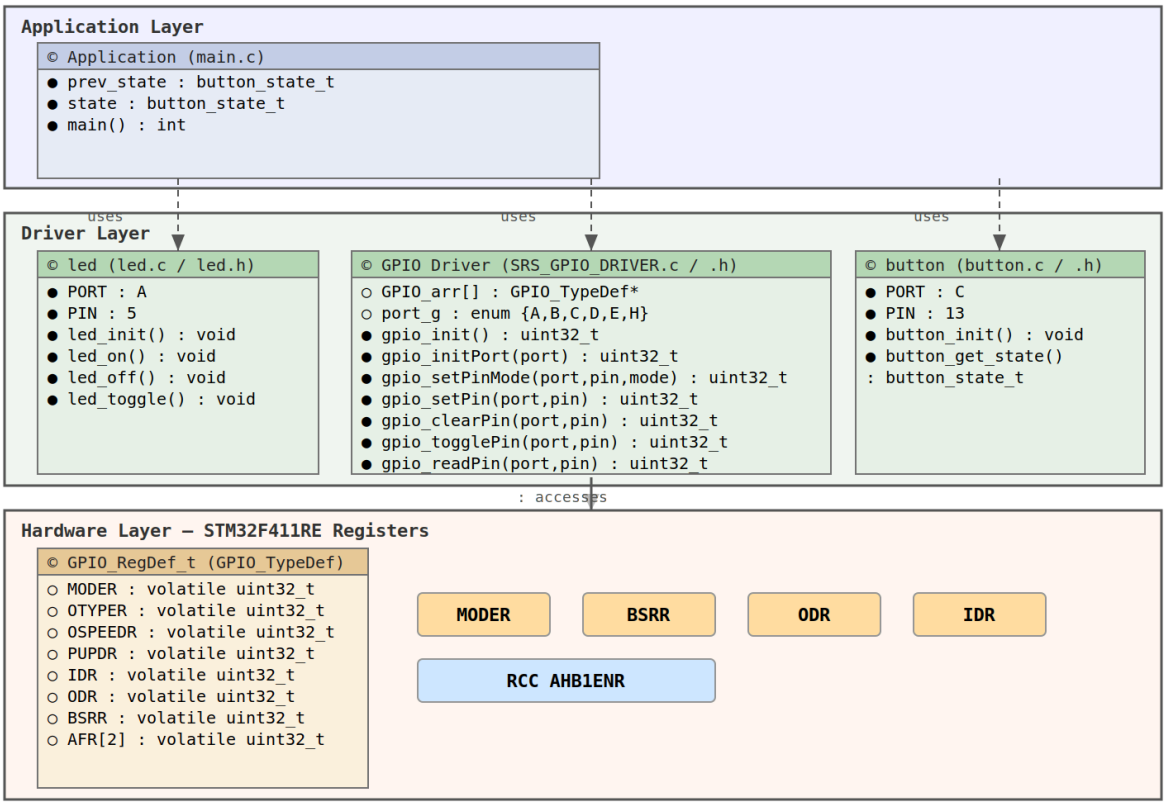

Figure 1. Class Diagram — GPIO Driver

Module Behavior

The following diagrams describe the behavioral design of each public function. Each function is covered by a sequence diagram (interaction between caller and driver) and an activity diagram (internal control flow including input validation).

gpio_init — Global Initialization (FR-1)

Populates GPIO_arr[] with CMSIS base addresses for all GPIO ports. Enables clocks for all six GPIO ports and resets the MODER register for ports B through H. Port A is excluded from the MODER reset to preserve the SWD debug interface pins PA13 and PA14. Must be called before any other GPIO driver function.

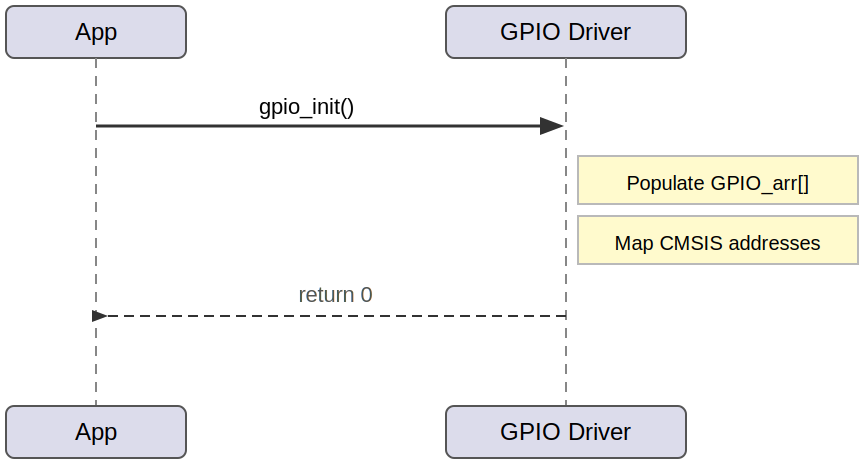

Figure 2. Sequence Diagram — gpio_init

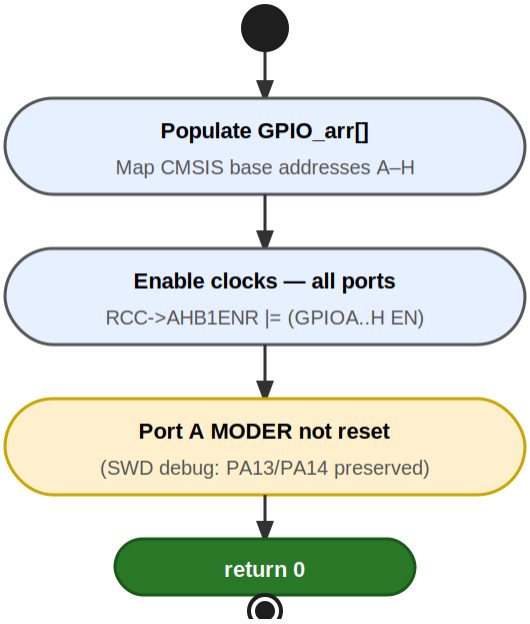

Figure 3. Activity Diagram — gpio_init

gpio_initPort — Per-Port Clock Enable (FR-2)

Validates the port parameter before enabling its peripheral clock selectively via the RCC AHB1ENR register. Returns an error code immediately if the port is out of range.

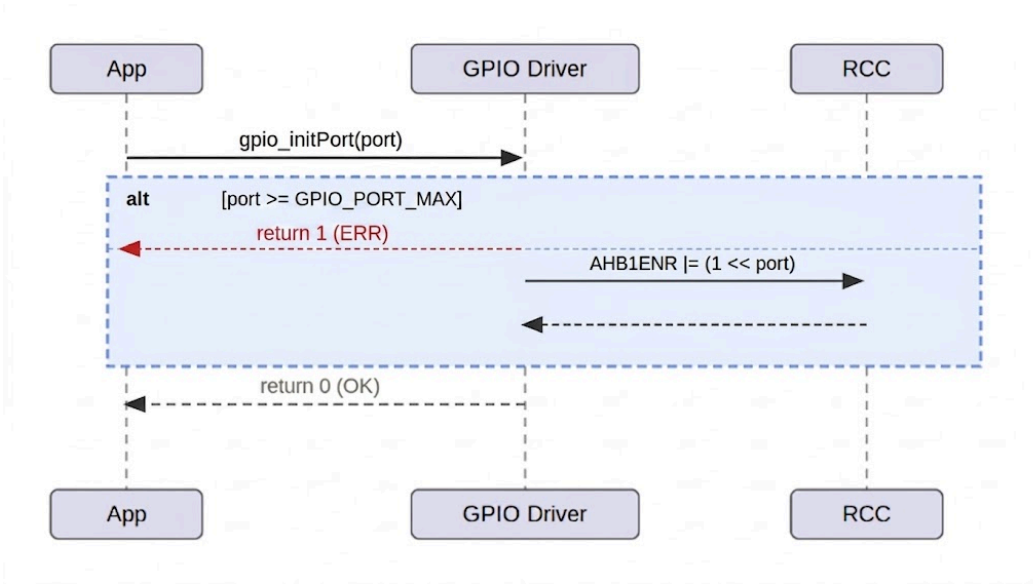

Figure 4. Sequence Diagram — gpio_initPort

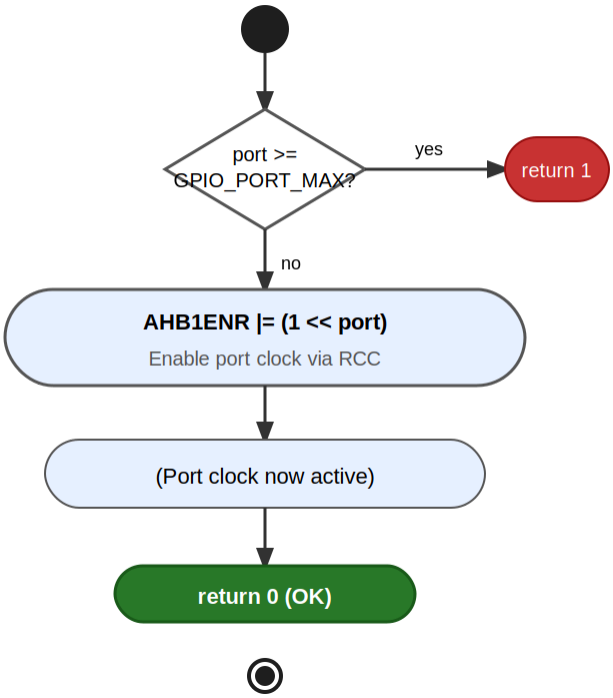

Figure 5. Activity Diagram — gpio_initPort

gpio_setPinMode — Pin Mode Configuration (FR-3)

Validates three parameters (port, pin, and mode) before performing a read-modify-write on the MODER register. The 2-bit field for the target pin is cleared first using MODER &= ~(0x3 << pin*2), then the new mode value is OR'd in using MODER |= mode << pin*2, leaving all other pin configurations intact.

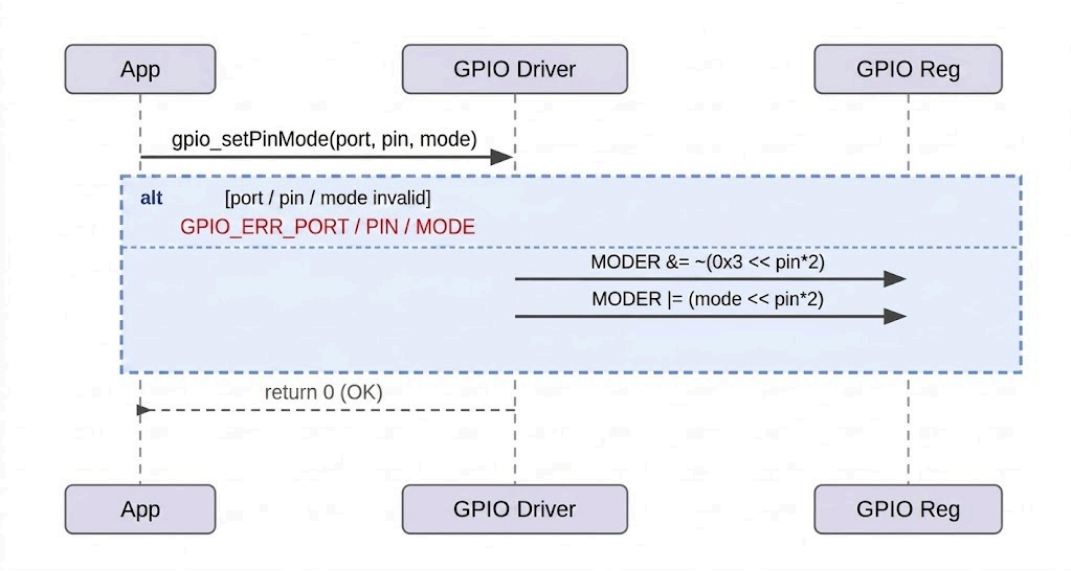

Figure 6. Sequence Diagram — gpio_setPinMode

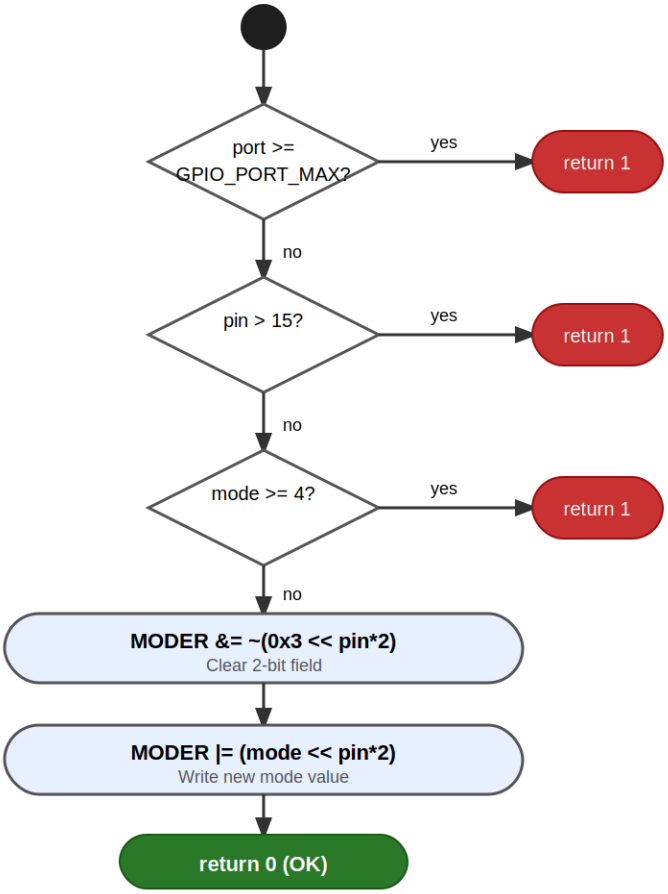

Figure 7. Activity Diagram — gpio_setPinMode

gpio_setPin / gpio_clearPin — Atomic Output Control (FR-4, FR-5)

Both functions write to the BSRR register, which guarantees atomic operation. gpio_setPin writes to the set-side (bits [15:0]) and gpio_clearPin writes to the reset-side (bits [31:16]), each requiring only a single write with no prior read.

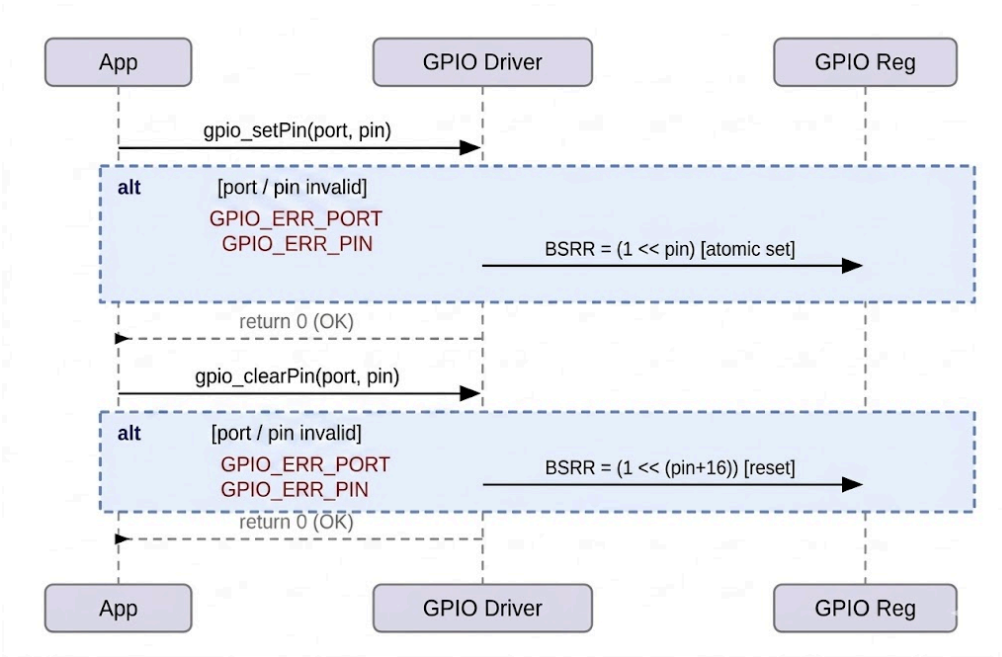

Figure 8. Sequence Diagram — gpio_setPin / gpio_clearPin

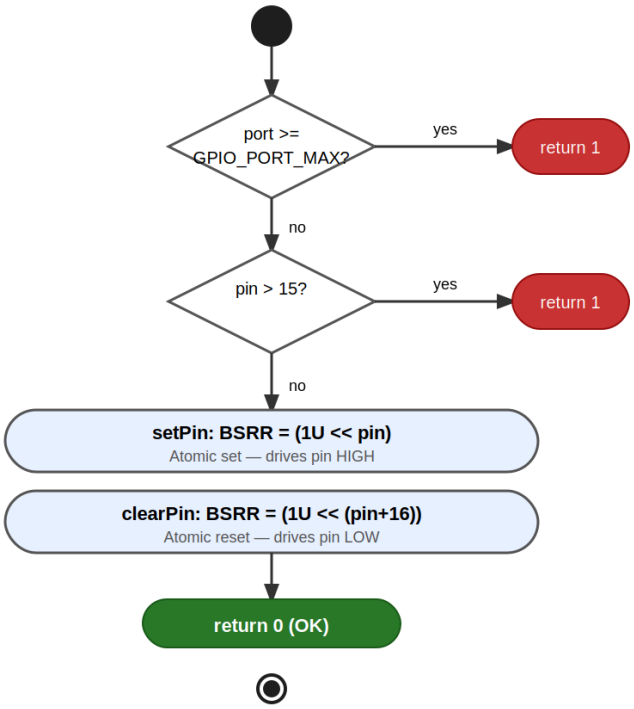

Figure 9. Activity Diagram — gpio_setPin / gpio_clearPin

gpio_togglePin — XOR Toggle (FR-6)

Reads the current ODR value and XORs bit pin to invert the output state (ODR ^= 1U << pin). This is a non-atomic read-modify-write operation on ODR. In the single-threaded polling loop used in this lab, mutual exclusion is not required. In an interrupt-driven design, the caller must ensure mutual exclusion.

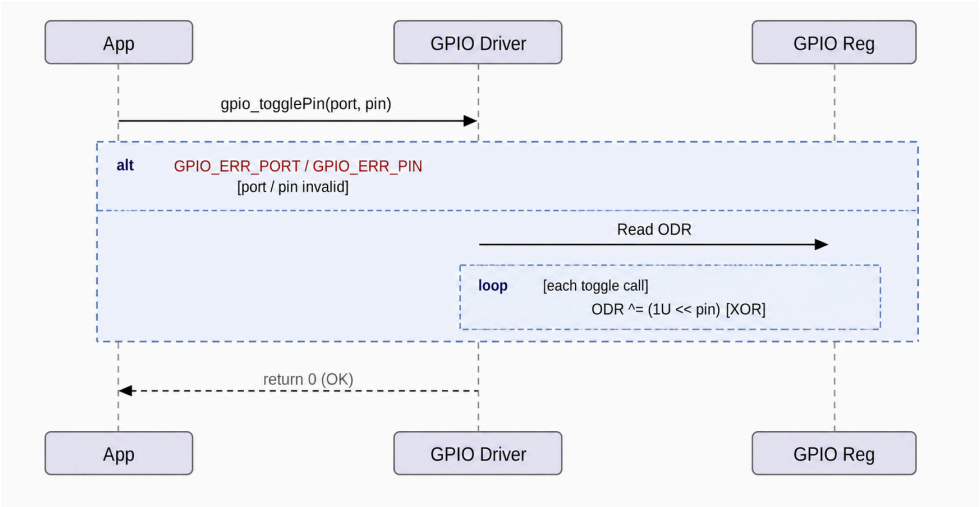

Figure 10. Sequence Diagram — gpio_togglePin

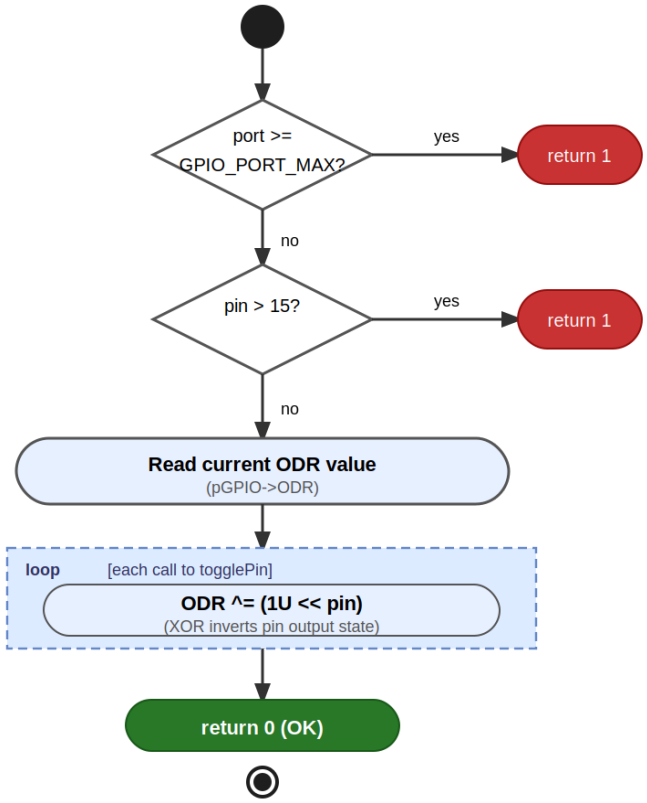

Figure 11. Activity Diagram — gpio_togglePin

gpio_readPin — IDR Read (FR-7)

Validates port and pin range before reading the IDR register. Extracts the single bit for the target pin using result = (IDR >> pin) & 0x1. The function works for both input and output configured pins since IDR always reflects the actual pin voltage.

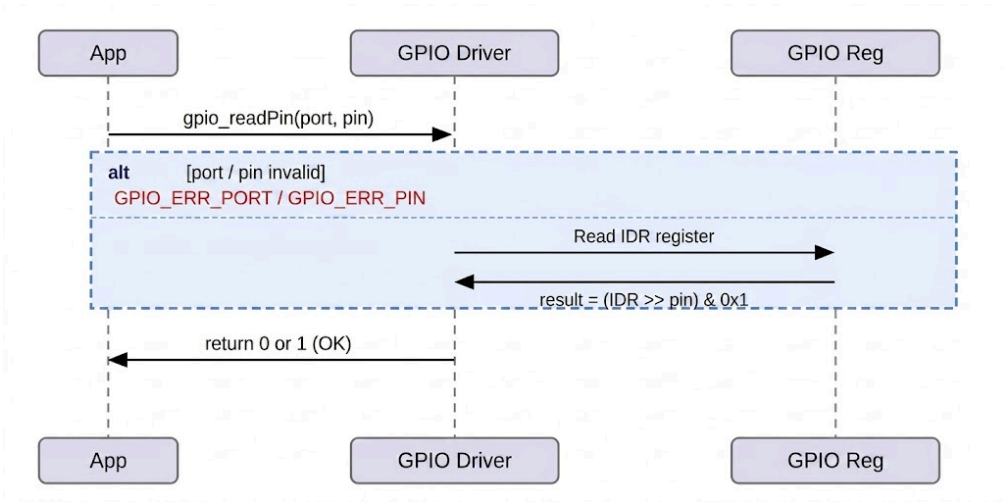

Figure 12. Sequence Diagram — gpio_readPin

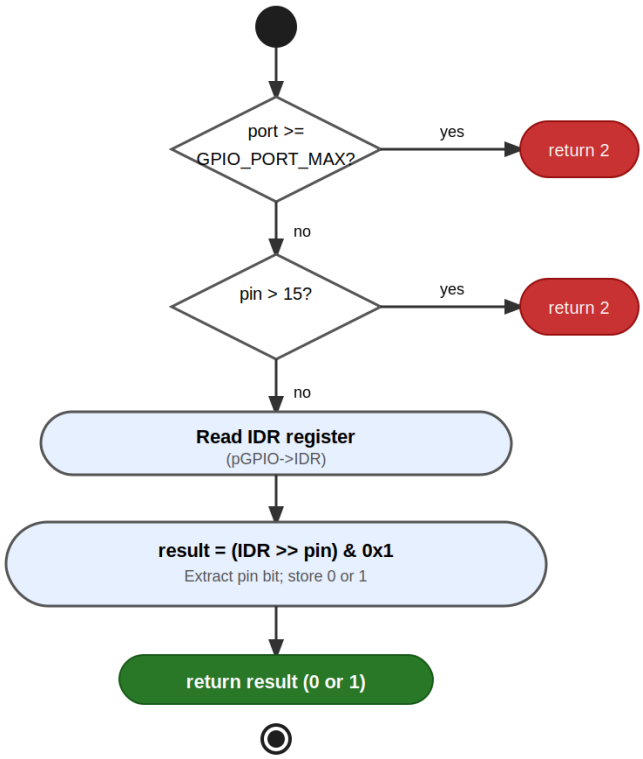

Figure 13. Activity Diagram — gpio_readPin

Driver State Machine

The state diagram below describes the overall lifecycle of the GPIO driver from system reset through initialization to normal application operation.

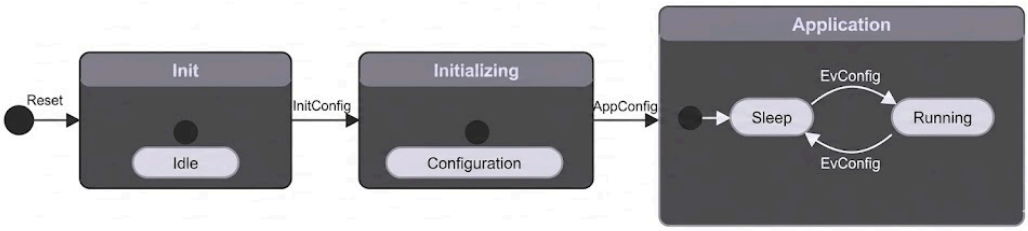

Figure 14. State Diagram — GPIO Driver Lifecycle

GPIO Driver

The GPIO driver for the STM32F411RE is implemented in two files: SRS_GPIO_DRIVER.h (public declarations and hardware definitions) and SRS_GPIO_DRIVER.c (implementation). The driver operates entirely through direct hardware register access — no HAL or vendor middleware is used.

Internal Data Structures

Two internal data structures are defined in SRS_GPIO_DRIVER.c and are not visible to external modules:

GPIO_arr[GPIO_PORT_MAX]:

An array of pointers to GPIO_TypeDef mapping each port_g enum value to its corresponding memory-mapped register block (GPIOA through GPIOH). All functions resolve a port by indexing this array rather than using conditional branching.

port_g — Port Enumeration:

An enum that maps human-readable port names (A through H) to integer indices. Acts as a validated port identifier throughout the driver. GPIO_PORT_MAX is a sentinel value used for bounds validation only and does not correspond to a physical port.

Initialization Flow

gpio_init (FR-1): Populates GPIO_arr[] with CMSIS base addresses for all supported ports. Enables peripheral clocks for all six ports simultaneously by OR-ing the corresponding RCC_AHB1ENR_GPIOxEN bits into RCC->AHB1ENR in a single write. Resets MODER to 0x00000000 for ports B, C, D, E, and H. Port A MODER is intentionally not reset to preserve SWD debug interface pins PA13 and PA14.

gpio_initPort (FR-2): Enables the clock for a single user-specified port by applying the corresponding bit mask from a clock mask lookup into RCC->AHB1ENR. Validates the port parameter before accessing any hardware register.

Pin Mode Configuration

gpio_setPinMode (FR-3) configures an individual pin mode in the MODER register. Each pin occupies a 2-bit field at bit position (pin × 2). The function first clears the field using MODER &= ~(0x3 << pin*2), then writes the new mode using MODER |= mode << pin*2. Three parameters are validated before any register access: port, pin (0–15), and mode (0–3).

Output Control — Set, Clear, Toggle

gpio_setPin (FR-4): Writes 1 to bit position pin in BSRR bits [15:0]. Atomic — no intermediate glitch state.

gpio_clearPin (FR-5): Writes 1 to bit position (pin+16) in BSRR bits [31:16]. Also atomic.

gpio_togglePin (FR-6): XORs bit pin in ODR using ODR ^= 1U << pin. Non-atomic read-modify-write — caller must ensure mutual exclusion under concurrent access.

Pin Read

gpio_readPin (FR-7) reads the IDR register, shifts right by pin positions, and masks with 0x1 to extract the target bit. The function works on both input and output configured pins since IDR always reflects the actual pin voltage.

LED Module

The LED module (led.c / led.h) provides a hardware abstraction for the onboard LED connected to PA5 on the NUCLEO-64 board. The module is restricted from accessing any hardware register directly and relies entirely on the GPIO driver API.

Active-low design note: On the NUCLEO-64, PA5 turns ON when driven LOW and OFF when driven HIGH. Therefore:

- led_on() calls gpio_clearPin(A, 5) — drives PA5 LOW → LED ON.
- led_off() calls gpio_setPin(A, 5) — drives PA5 HIGH → LED OFF.
- led_toggle() calls gpio_togglePin(A, 5) — inverts PA5 via XOR on ODR; no manual state tracking required.

Function	GPIO Call	Hardware Effect
led_init()	gpio_initPort(A) + gpio_setPinMode(A, 5, OUTPUT)	Enables GPIOA clock, configures PA5 as digital output.
led_on()	gpio_clearPin(A, 5)	PA5 = LOW → LED ON (active-low).
led_off()	gpio_setPin(A, 5)	PA5 = HIGH → LED OFF.
led_toggle()	gpio_togglePin(A, 5)	PA5 inverted → LED state flips.

Push-Button Module

The Push-Button module (button.c / button.h) encapsulates the read logic for the user button on PC13. Like the LED module, it accesses hardware exclusively through the GPIO driver API.

Active-low design note: The NUCLEO-64 board has an internal pull-up resistor on PC13. When the button is not pressed, PC13 is pulled HIGH (IDR = 1). When pressed, the pin is pulled LOW (IDR = 0). Therefore button_get_state() interprets:

- IDR = 0 → BUTTON_PRESSED
- IDR = 1 → BUTTON_RELEASED

Function	GPIO Call	Return Value
button_init()	gpio_initPort(C) + gpio_setPinMode(C, 13, INPUT)	void — configures PC13 as digital input.
button_get_state()	gpio_readPin(C, 13)	BUTTON_PRESSED (0) or BUTTON_RELEASED (1).

Application Integration — Edge Detection and Debounce

The Application Layer (main.c) integrates both modules inside an infinite polling loop. A software edge-detection strategy is used: the LED toggles only when the current state is BUTTON_PRESSED and the previous state was BUTTON_RELEASED (falling edge detection). This ensures that one physical press-and-release cycle produces exactly one LED toggle event, regardless of contact bouncing duration.

Variable	Type	Role
state	button_state_t	Current button reading from button_get_state() on each loop iteration.
prev_state	button_state_t	State stored from the previous iteration. Initialized to BUTTON_RELEASED.

Application Programming Interfaces

Data Types and Macros

Register-Map Structures

GPIO_TypeDef: Maps the memory layout of a single GPIO port's hardware registers. Each field is declared volatile to prevent compiler optimization of hardware register accesses.

Member	Type	Notes / Definition
MODER	volatile uint32_t	Offset 0x00 — configures each pin: input(00), output(01), alt-fn(10), analog(11).
OTYPER	volatile uint32_t	Offset 0x04 — output type: push-pull(0) or open-drain(1).
OSPEEDR	volatile uint32_t	Offset 0x08 — controls slew rate / output speed per pin.
PUPDR	volatile uint32_t	Offset 0x0C — configures pull-up(01) or pull-down(10) resistors.
IDR	volatile uint32_t	Offset 0x10 — read-only; reflects the current logic level on each pin.
ODR	volatile uint32_t	Offset 0x14 — read/write; controls output state of each pin.
BSRR	volatile uint32_t	Offset 0x18 — write-only; bits[15:0] set pins, bits[31:16] reset pins atomically.
AFR[2]	volatile uint32_t	Offset 0x20 — alternate function: AFR[0] for pins 0–7, AFR[1] for pins 8–15.

Enumerations

port_g: Identifies each supported GPIO port. GPIO_PORT_MAX is a sentinel value used for bounds validation only.

Enumerator	Value	Definition
GPIO_PORT_A	0	Port A

GPIO_PORT_B	1	Port B
GPIO_PORT_C	2	Port C
GPIO_PORT_D	3	Port D
GPIO_PORT_E	4	Port E
GPIO_PORT_H	5	Port H
GPIO_PORT_MAX	6	Sentinel — boundary validation; not a valid port.

Error Codes

All public GPIO API functions have a return type of uint32_t. Error codes are listed below:

Code	Value	Definition
0	0	Operation completed successfully.
1	1	The port parameter is invalid (>= GPIO_PORT_MAX).
1	1	The pin parameter is invalid (> 15).
1	1	The mode parameter is invalid (>= 4).
2	2	A required pointer is NULL (applies to gpio_readPin only).

Function Definitions

gpio_init

Function Name	gpio_init
Syntax	uint32_t gpio_init(void)
Parameters (in)	None
Parameters (inout)	None
Parameters (out)	None
Return value	0 on success.
Description	Populates GPIO_arr[] with CMSIS base addresses for all supported GPIO ports (A, B, C, D, E, H). Enables peripheral clocks for all six ports simultaneously by setting the corresponding bits in RCC->AHB1ENR. Resets the MODER register to 0x00000000 for ports B, C, D, E, and H. Port A MODER is intentionally not reset to preserve the SWD debug interface on PA13 and PA14.
Constraints	Must be called before any other GPIO driver function. Port A MODER is not reset.
Available via	SRS_GPIO_DRIVER.h

gpio_initPort

Function Name	gpio_initPort
Syntax	uint32_t gpio_initPort(port_g port)

Parameters (in)	port: port_g — selects the GPIO port whose clock will be enabled (GPIO_PORT_A through GPIO_PORT_H).
Parameters (inout)	None
Parameters (out)	None
Return value	0 on success. 1 if port >= GPIO_PORT_MAX.
Description	Enables the peripheral clock for a single GPIO port by setting the corresponding bit in RCC->AHB1ENR. Validates the port parameter before any hardware access.
Constraints	Does not reset or configure pin modes; only enables the clock. The caller must call gpio_setPinMode separately to configure individual pins.
Available via	SRS_GPIO_DRIVER.h

gpio_setPinMode

Function Name	gpio_setPinMode
Syntax	uint32_t gpio_setPinMode(port_g port, uint32_t pin, uint32_t mode)
Parameters (in)	port: port_g — target GPIO port.
Parameters (in)	pin: uint32_t — target pin number (0–15).
Parameters (in)	mode: uint32_t — desired pin mode (0=input, 1=output, 2=altfn, 3=analog).
Parameters (inout)	None
Parameters (out)	None
Return value	0 on success. 1 if port, pin, or mode is invalid.
Description	Configures the mode of an individual GPIO pin by writing to the 2-bit field in MODER at bit position (pin × 2). Clears the field first using MODER &= ~(0x3 << pin*2), then sets it using MODER |= mode << pin*2. All three parameters are validated before any register access.
Constraints	Does not configure PUPDR, OTYPER, or AFR. The port clock must be enabled before calling this function.
Available via	SRS_GPIO_DRIVER.h

gpio_setPin

Function Name	gpio_setPin
Syntax	uint32_t gpio_setPin(port_g port, uint32_t pin)
Parameters (in)	port: port_g — target GPIO port.
Parameters (in)	pin: uint32_t — target pin number (0–15).
Parameters (inout)	None
Parameters (out)	None
Return value	0 on success. 1 if port or pin is invalid.

Description	Atomically drives a GPIO output pin high by writing 1 to bit position pin in BSRR bits [15:0]. The BSRR register guarantees an atomic operation with no intermediate glitch state.
Constraints	The pin must be configured as output before calling. Port clock must be enabled.
Available via	SRS_GPIO_DRIVER.h

gpio_clearPin

Function Name	gpio_clearPin
Syntax	uint32_t gpio_clearPin(port_g port, uint32_t pin)
Parameters (in)	port: port_g — target GPIO port.
Parameters (in)	pin: uint32_t — target pin number (0–15).
Parameters (inout)	None
Parameters (out)	None
Return value	0 on success. 1 if port or pin is invalid.
Description	Atomically drives a GPIO output pin low by writing 1 to bit position (pin + 16) in BSRR bits [31:16]. The operation does not require reading ODR first.
Constraints	The pin must be configured as output before calling. Port clock must be enabled.
Available via	SRS_GPIO_DRIVER.h

gpio_togglePin

Function Name	gpio_togglePin
Syntax	uint32_t gpio_togglePin(port_g port, uint32_t pin)
Parameters (in)	port: port_g — target GPIO port.
Parameters (in)	pin: uint32_t — target pin number (0–15).
Parameters (inout)	None
Parameters (out)	None
Return value	0 on success. 1 if port or pin is invalid.
Description	Inverts the current output state of a GPIO pin by XOR-ing bit pin in the ODR register (ODR ^= 1U << pin). The resulting state depends on the current ODR value.
Constraints	Non-atomic read-modify-write on ODR. In the single-threaded polling loop of this lab, mutual exclusion is not required. In an interrupt-driven design, the caller must ensure mutual exclusion.
Available via	SRS_GPIO_DRIVER.h

gpio_readPin

Function Name	gpio_readPin
Syntax	uint32_t gpio_readPin(port_g port, uint32_t pin)

Parameters (in)	port: port_g — target GPIO port.
Parameters (in)	pin: uint32_t — target pin number (0–15).
Parameters (inout)	None
Parameters (out)	None
Return value	0 or 1 (pin logic level) on success. 2 if port or pin is invalid.
Description	Reads the current logic level of a GPIO pin from the IDR register by shifting right by pin and masking with 0x1. Returns the result directly as the function return value. Works on both input and output configured pins since IDR always reflects the actual pin voltage.
Constraints	The port clock must be enabled.
Available via	SRS_GPIO_DRIVER.h

Internal Function Definitions

The GPIO driver does not define any internal (static) helper functions. All logic is contained within the public API functions. Internal state is maintained exclusively through the GPIO_arr[] pointer array described in Section 3. No internal functions are subject to this section.

Future Enhancements

The following enhancements may be considered for future revisions of this GPIO driver:

- gpio_setAlternateFunction: Implement alternate function configuration by writing to the AFR[0] and AFR[1] register fields. This is required to enable peripheral functions such as UART, SPI, and I2C on GPIO pins.
- Output type configuration (OTYPER): Add a function to configure a pin as push-pull or open-drain output.
- Pull-up / pull-down configuration (PUPDR): Add a function to set or clear internal pull-up and pull-down resistors per pin.
- Output speed configuration (OSPEEDR): Add a function to control the slew rate of output pins (low, medium, high, very high speed).
- Interrupt support (EXTI): Add support for configuring GPIO pins as external interrupt sources, including edge selection (rising, falling, both) and enabling the NVIC for the corresponding EXTI lines.
- Port-level read / write operations: Add functions to read or write all 16 pins of a port simultaneously using IDR and ODR.
- Configuration locking (LCKR): Add a function to lock the configuration of selected pins using the LCKR register, preventing accidental reconfiguration after initialization.
- Thread-safety improvements: Introduce critical section wrappers around the non-atomic ODR read-modify-write in gpio_togglePin.